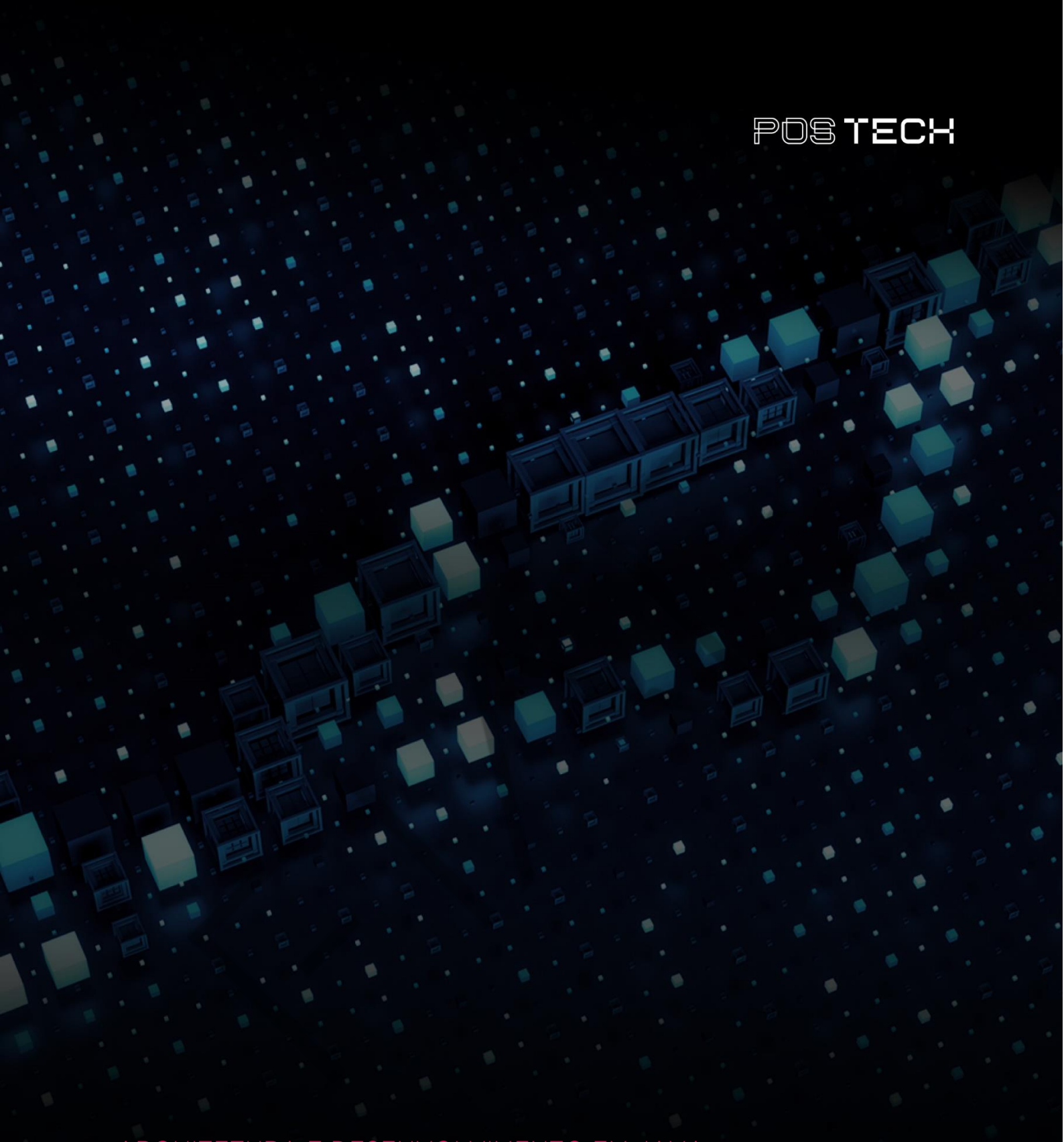


ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 4 | AULA 01 -

INTRODUÇÃO À COMPUTAÇÃO SERVERLESS

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 13
REFERÊNCIAS 14

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Criar soluções serverless não significa apenas criar aplicações rodando em nuvem, com uma quantidade grande de funções como serviço interligadas por algum tipo serviço de eventos. Essas ferramentas são parte da solução e, se bem usadas, podem criar ótimas soluções, mas elas não são, em si, uma solução de arquitetura serverless.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Vamos entender melhor quais as ideias por trás da arquitetura serverless, porque esse tipo de arquitetura tem sido discutida como uma saída viável para a construção de software e quais os benefícios e desafios que esta abordagem apresenta.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

O que é o serverless?

Uma definição para serverless pode ser simples e complexa ao mesmo tempo. Podemos definir tecnicamente como o serverless funciona, mas não conseguimos, olhando apenas para os componentes técnicos, definir o que é o serverless como arquitetura.

Numa definição simples, podemos dizer que a arquitetura serverless é aquela criada para funcionar sem servidores, ou pelo menos sem a preocupação com o provisionamento de servidores no nível da arquitetura da solução. Ainda assim, temos provisionamento, mas de estruturas de serviços interconectadas, fazendo com que esses serviços funcionem corretamente em conjunto.

O ambiente cloud

Pensamos nessa solução em termos de componentes e serviços, do mesmo jeito que fazemos na arquitetura de soluções que conhecemos atualmente. A diferença é que o software não vai rodar em um servidor previamente definido, que precisa ser criado e configurado. Também não existe a necessidade de construir uma estrutura de servidores com serviços como banco de dados, filas e armazenamento, pois todos esses serviços são consumidos de um provedor conforme o uso, e não conforme a disponibilidade.

Passamos, então, de uma arquitetura onde a aplicação funcionava com servidores e toda a estrutura gerenciada pelo time da empresa (Figura 1) para outra possibilidade: rodar a aplicação sem precisar criar e gerenciar os servidores diretamente (Figura 2).

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

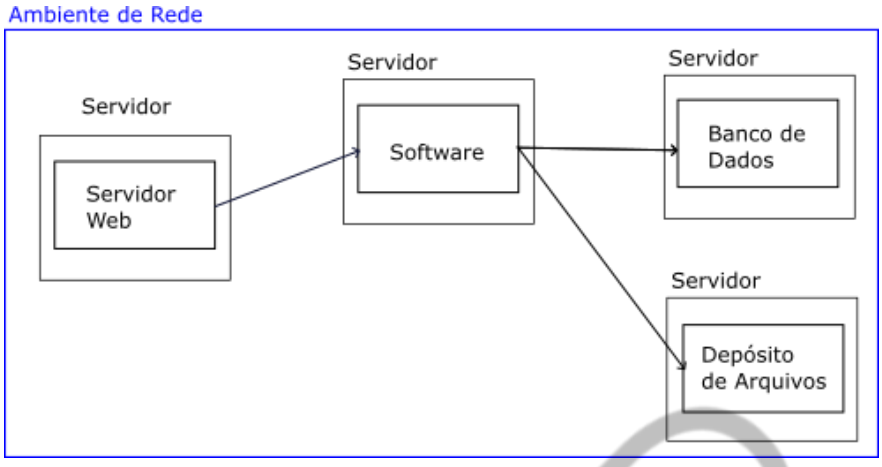


Figura 1 – Estrutura gerenciada pela empresa

Fonte: elaborado pelo autor (2025)

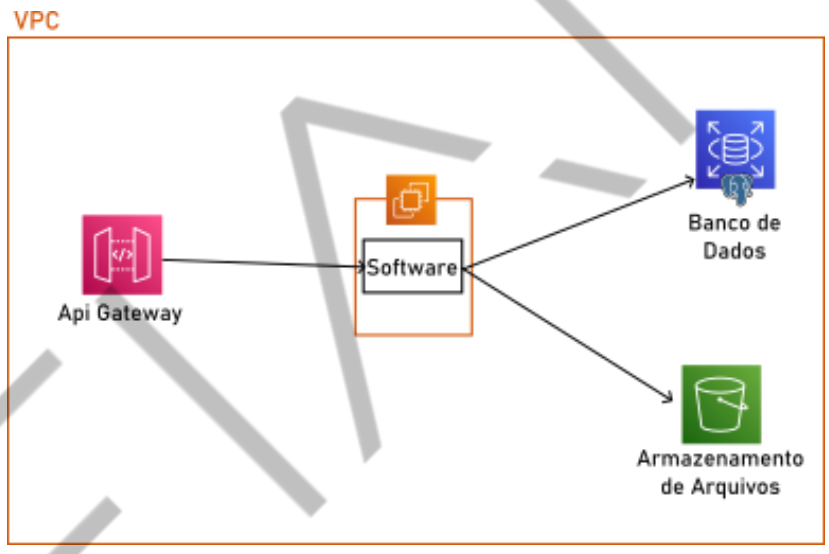


Figura 2 – Estrutura sem gerenciamento direto

Fonte: elaborado pelo autor (2025)

Nas figuras, temos diferentes ícones representando cada um dos serviços, com base nos serviços existentes na AWS. Ao invés de um servidor para validar e rotear as chamadas, temos um serviço que realiza essa função (API Gateway). Ao invés de um servidor para a aplicação, temos uma instância de processamento rodando a aplicação usando outro serviço (EC2) e temos o RDS para o banco de dados e o S3 para o armazenamento de arquivos. Tudo isso dentro de um ambiente de rede configurado (VPC).

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Não é preciso criar os servidores “à mão”, apenas criar uma instância do serviço, configurar e usar. No caso do banco de dados, por exemplo, todo o trabalho de configurar os parâmetros do servidor do banco de dados da tecnologia escolhida (por exemplo, Postgre) é delegada ao gerenciador do serviço. Para criar a solução, apenas configuramos a instância, informando quantidade de memória disponível, espaço de armazenamento, e assim por diante.

Essa visão de integração de serviços só foi possível ser adotada em larga escala com o surgimento dos provedores de serviços em nuvem (ambientes Cloud) e serviços de plataforma, como SaaS, PaaS ou IaaS. Porque, agora, não gerenciamos mais os servidores, pagamos apenas pelo uso, também não é necessário ter um Data Center, já que é possível hospedar o software em um ambiente segregado, seguro e garantido. Ao pagar pelo uso, é possível que as soluções não tenham, a princípio, um compromisso financeiro prévio.

O desafio da escalabilidade

Com todas as características acima, no momento da arquitetura, podemos começar a pensar em usar os serviços, não em configurar os servidores. Além disso, ganhamos também escala em uso, pois é possível dimensionar a infraestrutura conforme necessário.

Mas há um problema: o software, que ainda não escala corretamente. Podemos escalar o servidor onde o software roda (escalabilidade vertical), podemos distribuir o processamento em diversos servidores (escalabilidade horizontal) ou podemos restringir o uso, mas essas soluções sempre trazem algum tipo de desafio na implementação, como concorrência por recursos, impossibilidade de escalabilidade automatizada e monitoramento, para citar alguns. Além de estarem atreladas a um servidor de aplicação, que ainda deve ser configurado.

Serviços e serverless

A ideia, então, foi “desmontar” o software em diversas partes, tomando como base alguns conceitos que já estavam sendo usados anteriormente, como a Arquitetura Orientada a Serviços (SOA) e os protocolos de comunicação entre serviços. A novidade foi criar pequenos pacotes de código que funcionem de forma autônoma e que se comuniquem para resolver o problema. Então, o software que

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

realizaria dez funcionalidades/funções poderia ter estas funções rodando dentro de um serviço próprio para a execução de código, as funções como serviço.

A ideia é interessante, pois permite que a escalabilidade não seja mais dos servidores, e sim da execução de cada uma dessas funções isoladas, possibilitando ajustar o ambiente de execução de cada uma delas e viabilizando alterar individualmente conforme necessário, sem impactar o resto da aplicação.

No exemplo abaixo, temos uma função de registro de compra que pode chamar diretamente uma função de cadastro de pessoas (caso a pessoa cliente não tenha cadastro). Posteriormente, ela pode criar um pedido de execução (via fila) para a função de finalização da compra que, por sua vez, pede a execução de uma função de registro de pagamento.

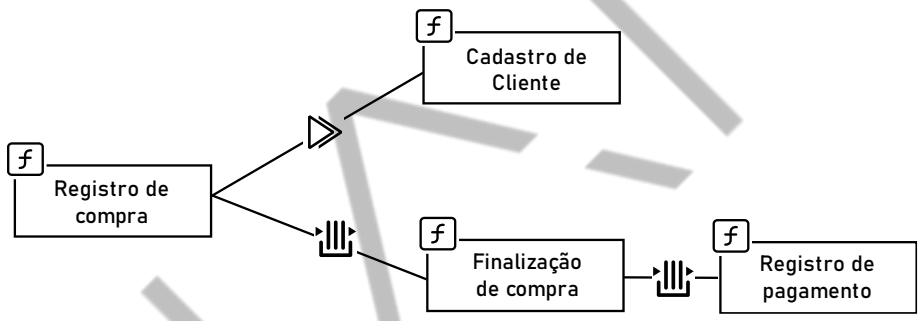


Figura 3 – Função de registro de compra
Fonte: elaborado pelo autor (2025)

Desta forma, pensamos na aplicação como uma coordenação de execuções e no modo como estas execuções se comunicam. De forma geral, o que se conhece como “arquitetura serverless” é, na verdade, a decomposição da aplicação em diferentes funções e o uso de serviços para permitir que estas funções atuem de forma independente. A ideia principal é permitir que todos os componentes de uma aplicação tenham uma função específica e rodem de forma isolada em relação aos outros componentes.

Com o aumento do uso das ideias dessa arquitetura, a construção de software passou a ser cada vez mais influenciada pelo conceito de “separação em funções serverless” como requisito de arquitetura, mais do que como solução, muito influenciado, também, pela noção de que não seria mais necessário construir uma aplicação completa ou monolítica, dado que era “melhor” que fosse distribuída em

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

diversos pontos de execução. Com isso, a ideia de arquitetura serverless se tornou o principal paradigma para a construção de software.

Quais problemas o serverless resolve?

Citando novamente os pontos já abordados, podemos entender que a arquitetura serverless resolve alguns problemas como:

Gerenciamento de infraestrutura substancialmente simplificado e terceirizado: uma das transformações mais significativas é a completa eliminação da necessidade de provisionar, configurar e gerenciar servidores de forma manual e contínua. A equipe de desenvolvimento é liberada dessas tarefas de baixo nível, podendo redirecionar foco e energia para o que realmente agrega valor ao negócio: a criação e aprimoramento de funcionalidades e a lógica da aplicação em si. Não existe mais a preocupação com toda a base operacional, já que ela está sendo atualizada e monitorada constantemente por uma equipe dedicada a manter a estrutura funcionando.

Escalabilidade: escala dinamicamente os recursos computacionais conforme necessário, de acordo com os parâmetros definidos, sem intervenção manual. Isso permite, e até mesmo garante, que a performance e a disponibilidade da aplicação não sejam degradadas conforme o aumento do uso, reduzindo também os custos de ociosidade. Com isso, temos não apenas uma maior resiliência e confiabilidade da aplicação, que adapta a quantidade de recursos às mudanças de carga sem sobrecarregar o time de desenvolvimento com o gerenciamento da infraestrutura de escalonamento, mas também uma otimização de custos significativa, pois paga-se apenas pelos recursos efetivamente consumidos.

Custos operacionais: como já falamos, o modelo de precificação é eficiente em custo, pois é baseado no consumo. Você paga apenas pela execução da função, ou seja, pelo tempo de execução real do código e pelos recursos utilizados durante esse período. Assim, não existem as despesas com servidores ociosos que ficariam aguardando requisições. Além disso, a responsabilidade pelo gerenciamento da infraestrutura, como manutenção e atualizações, é do provedor de serviços. Consequentemente, há uma diminuição nos gastos com mão de obra especializada para essas tarefas, otimizando o orçamento de TI.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Time-to-market: as pessoas que criam o código concentram seus esforços exclusivamente na criação do código e na criação das funcionalidades essenciais do negócio, sem dedicar tempo à administração de servidores, balanceamento de carga ou configurações de rede. Isso permite a implementação de metodologias ágeis, possibilitando ciclos de desenvolvimento e entrega mais curtos. A simplicidade de implantação também reduz barreiras operacionais, permitindo que novas versões e funcionalidades sejam disponibilizadas aos usuários frequentemente.

Desacoplamento de aplicações: como a arquitetura frequentemente é associada a microsserviços e funções como serviço (FaaS), ela incentiva o desenho e o desenvolvimento de componentes menores e independentes, facilitando o desacoplamento das partes de uma aplicação, tornando-a mais fácil de manter e permitindo que diferentes equipes trabalhem em paralelo.

Que problemas o serverless causa?

Contudo, não existe “bala da prata” e a implementação de serverless apresenta diversos desafios. Alguns deles são:

Complexidade arquitetural: gerenciar um grande número de funções serverless interdependentes pode se tornar desafiador, criando uma versão atualizada do “código espaguete” (que é difícil de manter), o “function spaghetti”. Manter a visão geral da arquitetura, entender as dependências, coordenar a execução das funções em cada fluxo de negócios e organizar as implantações pode exigir um esforço considerável. Planejar o ciclo de vida das aplicações também tende a ser um desafio, porque temos uma facilidade de implementação de funções e isso pode criar uma profusão de novas funções muitas vezes redundantes ou simplesmente desnecessárias, criadas apenas para resolver um problema pontual, contaminando os projetos e arquiteturas.

Complexidade de monitoramento e debugging: o monitoramento torna-se mais complexo devido à natureza distribuída e efêmera (execução curta, sem estado interno) das funções, dificultando o rastreamento de problemas e a análise de desempenho. Entender o processo de execução de uma regra de negócio pode se tornar uma tarefa muito complexa, sendo necessário cruzar diferentes informações de log, tempo de execução e valores de entrada e saída. Muitas vezes, até mesmo fazer uma depuração de execução pode ser uma tarefa praticamente impossível. Mesmo

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

com ferramentas de rastreamento, que hoje estão disponíveis, é muito difícil acompanhar a execução de alguma função.

Dificuldades nos testes locais: replicar o ambiente exato da nuvem serverless em uma máquina de desenvolvimento local para testes pode ser difícil, tornando testes consistentes entre ambientes de desenvolvimento e produção praticamente impossíveis. Isso ocorre porque muitos dos recursos usados em ambiente serverless são específicos do contexto de execução, como simular gatilhos de execução, validar e aplicar permissões de IAM e integrações com outros serviços na nuvem. Tudo isso adiciona complexidade ao processo de teste.

Custos (possivelmente) imprevisíveis: embora o modelo "pague pelo uso", ou “pay-as-you-go” seja geralmente uma vantagem, ele também pode levar a custos inesperados se as funções forem acionadas com mais frequência do que o previsto, se ficarem presas em loops de invocação ou, ainda, se mal dimensionadas ou com problemas de execução. Se o custo de um recurso é variável, não é possível fazer um planejamento de investimento ou gasto prévio, porque mesmo as estimativas podem ser impactadas por eventos inesperados.

Limitações e restrições da plataforma: os provedores de nuvem impõem limites aos serviços serverless para execução de funções, como tempo máximo de execução de uma função (por exemplo, 15 minutos na AWS Lambda), limites de memória, tamanho do pacote de código e concorrência de execuções. Aplicações com processos de longa duração (por exemplo, servidor de aplicação que precisa estar disponível para atender a requisições, como Spring) ou requisitos de recursos muito específicos (processamento complexo de grandes volumes de dados) podem não se adequar bem a essas restrições.

Cloud native

Com o aumento do uso e disponibilidade de ferramentas de nuvem, emergiu um modo diferente de pensar a criação das soluções, que hoje é conhecido como cloud native. Uma aplicação cloud native é projetada para existir e operar em ambientes de nuvem, aproveitando integralmente os benefícios e características que estas plataformas oferecem. O termo "cloud native" refere-se a uma abordagem que assume a nuvem como ambiente principal desde o desenho do projeto, em oposição

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

a aplicações tradicionais que são adaptadas ou migradas posteriormente para operar em nuvem.

As aplicações cloud native são tipicamente estruturadas como microsserviços, componentes independentes e fracamente acoplados que podem ser desenvolvidos, atualizados e escalados de maneira autônoma. Estas aplicações são imutáveis, o que significa que não são modificadas após a implantação, mas substituídas por novas versões quando necessário. São projetadas para serem resilientes, automaticamente recuperando-se de falhas sem intervenção manual. Também existe um processo de migração do deploy para ferramentas automatizadas, para que a implantação seja segura e rápida, mas focada na publicação de pequenas funções, tornando os ciclos de desenvolvimento mais ágeis, reduzindo riscos e acelerando a entrega de valor.

Para pessoas arquitetas, o desafio é projetar sistemas distribuídos, pensando em resiliência e comunicação eficiente. Nesta abordagem, a arquitetura de software começa a priorizar, desde o início, princípios como comunicação entre componentes e serviços, desacoplamento, isolamento de falhas e elasticidade, favorecendo designs distribuídos. Escalabilidade, segurança em ambientes complexos e observabilidade são preocupações novas para a arquitetura da solução, dado que são fornecidas por serviços, que precisam ser integrados à solução.

Então a expressão "cloud native" representa uma abordagem focada na arquitetura da aplicação do começo ao fim, não apenas na criação do software. Visa construir e executar aplicações que não apenas operam, mas se beneficiam da estrutura na nuvem, aproveitando sua elasticidade e automação. Isso vai além de migrar sistemas legados; trata-se de uma mentalidade de "nascer na nuvem", integrando desde o início tecnologias e processos ágeis e definindo a aplicação como um conjunto de módulos bem organizados e com comunicação eficaz. Ao final, a meta é criar soluções robustas, flexíveis e que extraiam o máximo valor das plataformas de nuvem, preparando as aplicações para evoluir constantemente e atender às exigências do negócio.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Se definir o que é serverless pode ser complicado ou até mesmo ser um desafio devido à quantidade de diferentes ideias que este conceito apresenta, conseguimos definir, de forma mais clara, quais são os componentes de arquitetura que nos ajudam a identificar se uma solução pode ser classificada como serverless.

Também entendemos que a visão da arquitetura serverless passa por uma mudança de paradigma que é profundamente vinculada ao uso de ferramentas disponibilizadas pelos provedores de nuvem e que, agora, é o momento de entender o próximo passo de arquitetura serverless: a construção de soluções “Cloud Native”.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

GOOGLE Cloud. **Modelo de arquitetura do Google Cloud**. Disponível em: https://cloud.google.com/architecture/framework?hl=pt-br. Acesso em: 07 jul. 2025.

INDRASIRI, K; SUHOTHAYAN, S. **Design Patterns for Cloud Native Applications**. [S.l.]: O'Reilly, 2021.

MICROSOFT. **Azure Architecture Center**. Disponível em: https://learn.microsoft.com/en-us/azure/architecture. Acesso em: 07 jul. 2025.

NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. [S.l.]: O'Reilly, 2021.

FIAP

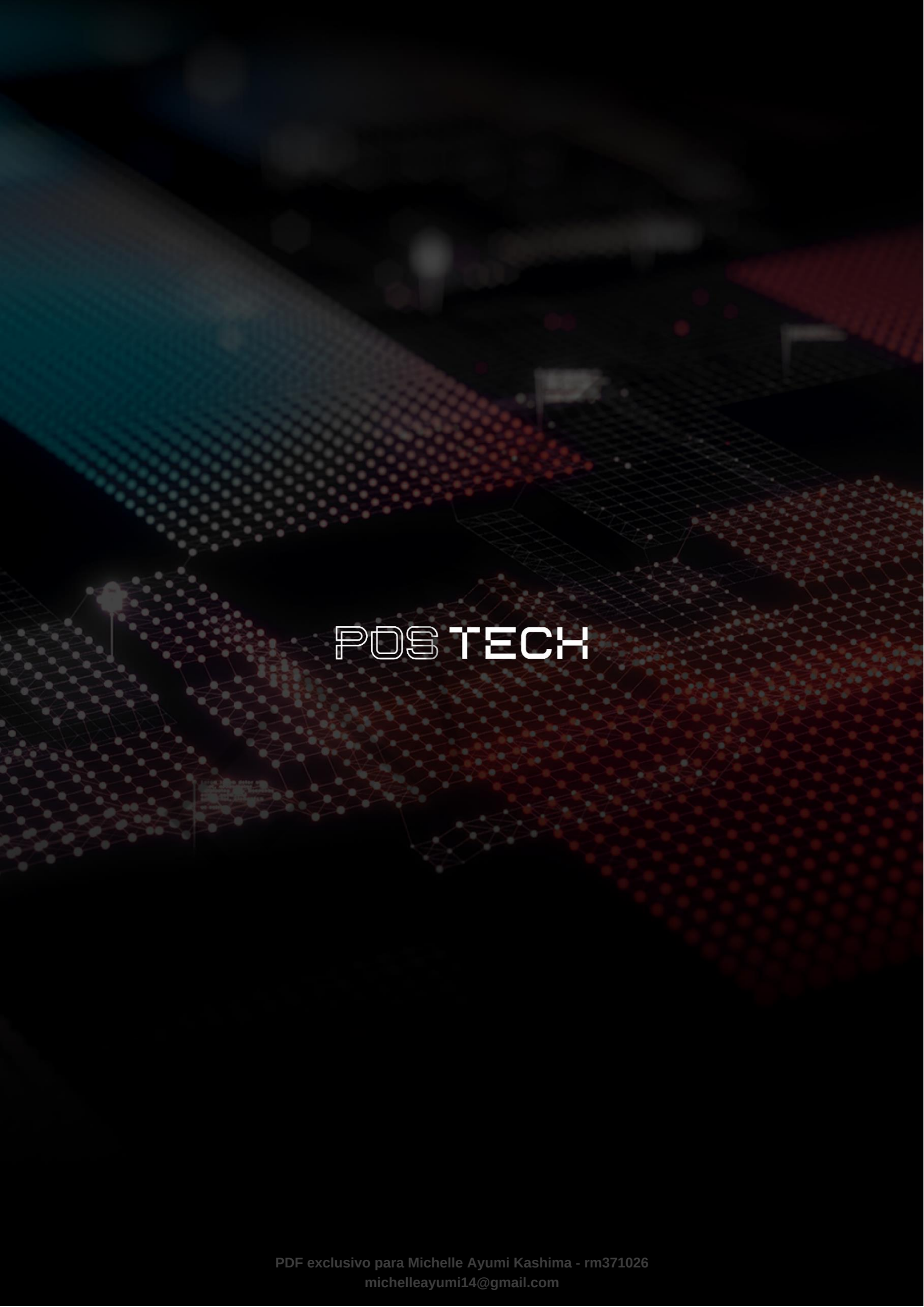
PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Arquitetura serverless. Cloud Native. Serverless.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com